

Response to reviewers — SOFTX-D-26-00952

We thank both reviewers. Reviewer 2's objection was that the evaluation could not support the claims without a direct comparison against contemporary GPU-enabled simulators, on realistic published workloads, under one benchmarking methodology. We agreed, and rather than argue we performed that work. It is the reason the manuscript is substantially different.

Section names below refer to the resubmitted manuscript.

Reviewer 1

1. The introduction must be revised and strengthened, with a more accurate description of the existing methods.

Rewritten. "Existing accelerated simulators, and what they ask of a user" now describes NEURON, CoreNEURON and Arbor in terms of what each actually does — CoreNEURON stripping the interpreter and running NEURON's models through OpenACC, Arbor written from scratch for many-core and GPU hardware with its own model description — and states plainly why NEST and Brian 2 are outside the comparison: they simulate networks of point neurons rather than dendritic morphologies, so they would not be running the same computation. The section ends by naming the gap this release addresses, which is not "fastest simulator" but acceleration without migration.

2. The contribution should be clearly stated in the introduction.

There is now an explicit "Contributions" list of five items, and it includes the head-to-head evaluation Reviewer 2 asked for.

3. There are formatting errors.

The manuscript was rebuilt against the SoftwareX Original Software Publication template. We added the CRediT authorship statement, the competing-interest declaration and the funding statement, which were missing; corrected the generative-AI declaration to the section title and placement the guide specifies; rewrote the highlights to five bullets within the 85-character limit; and raised Figure 1 to the resolution the guide requires, which it did not meet.

4. What are the main advantages of GPU acceleration here?

Addressed in "Why a GPU helps here is specific to the workload": in a Hodgkin-Huxley model the per-compartment gating integration dominates the cost and is embarrassingly parallel, each compartment advancing from its own voltage and its own table lookups. The state is small and the arithmetic intensity low, so the work maps onto thousands of lightweight threads. We also say where it does not help, measured: for a single detailed cell only one thread is active and the GPU is some 20x slower than the CPU.

5. What is the role and significance of OpenCL and CUDA?

Addressed in "We provide both backends because they buy different things". OpenCL is vendor-neutral and runs on the integrated GPUs shipped with ordinary workstations, including devices without double precision, which is why the kernel is fp32 throughout; CUDA targets the data-center cards where cluster time is available. The manuscript now also reports what the choice costs: the OpenCL backend trails CUDA by roughly 8%.

6. What is the impact of the GPU tridiagonal (Hines) elimination kernel?

It is what decides whether any of this reaches realistic morphologies, and the manuscript says so directly. Accelerating channel updates alone leaves axial coupling on the CPU, where the solve becomes the bottleneck as soon as the channel work is removed from it. Table 3 and Figure 2 quantify what the kernel achieves and, separately, how much of it survives end-to-end.

7. More discussion of future study, at higher dimension, in the conclusion.

The future-work section was rewritten around three limits we measured rather than anticipated. The first is exactly the scaling question: the kernel gives each tree one thread, so reaching the single detailed morphology — the regime much of the literature works in — requires parallelism inside the tree, such as cyclic reduction. We name that as where the design is furthest from complete.

10. Please pay special attention to cite the references, followed by two suggested works on synchronization of Markovian jump neural networks with application to image encryption, and on sampled-data control of semi-Markovian jump competitive neural networks in power systems.

We read both. Neither bears on compartmental neural simulation, GPU acceleration of Hodgkin-Huxley kinetics, or the Hines solver, and we could not cite them in a way that would inform a reader of this paper. We have instead strengthened the citations where they carry weight: the simulators we compare against, the benchmark model, and the Hines method itself. We hope the reviewer will accept this.

Reviewer 2

Novelty and practical significance cannot be assessed without direct comparison against CoreNEURON or Arbor.

Both are now in the manuscript, on two models and on both processors.

On the Vogels-Abbott COBAHH network (Table 5, Figure 4): GENESIS 2.5 against NEURON 9.0.2, CoreNEURON on CPU, CoreNEURON on an A100, and Arbor on an A100. On a 160,000-compartment multi-compartment population (Table 6, Figure 5): GENESIS 2.5 against NEURON and Arbor, GPU against GPU.

We report the outcomes that do not favour us. CoreNEURON on an A100 runs the spiking benchmark 1.23x faster than our best arm. Against Arbor neither simulator is faster in general: the two are linear in run length with different intercepts and slopes, so their lines cross, and a comparison at a single run length reports whichever side of the crossing its author chose.

The validation is narrow: controlled Hodgkin-Huxley benchmarks and simple structures, with validation on established PGENESIS network models left as future work.

That is no longer future work. The Vogels-Abbott COBAHH network (ModelDB 83319) is a published benchmark with a reference NEURON implementation, and it is now the manuscript's principal cross-simulator model. Running it also exposed a limit in GENESIS itself: hsolve tracked one spike generator per solver, which forced such a network to be built as one solver per cell. This release lifts that — a whole layer becomes a single solver, 1.41x faster on the CPU alone — and the accelerator computes a synaptically coupled spiking network for the first time.

We state its outcome honestly: on the device that network runs at parity with its own CPU arm, not faster, because a zero-delay network cannot batch dispatches. The manuscript explains the mechanism and what would have to change.

Performance comparisons use different simulation lengths, hardware and timing approaches; the largest speedups concern the simulation step rather than complete

execution; performance is unfavourable for small workloads.

This was a fair criticism and the methodology is now uniform.

One timing definition. Every ratio wall-clocks both arms around the whole process, construction and transfers included. The manuscript states this explicitly and says why it matters: timing the two arms by different clocks inflates the ratio roughly tenfold and can invent differences between cards.

Step-phase reported separately and labelled. Where we give a step-phase figure we give the end-to-end figure beside it and say which is which — for example 80.8x step-phase against 3.90x end-to-end at K=200 on the A100 — and we write that quoting only the step-phase number would overstate what the accelerator delivers.

Run length treated as a variable, not a choice. Because the end-to-end figure depends on how long the run is, we measured that dependence (Table 4) instead of picking a length: the speedup rises from 4.3x at K=200 to 57x at K=5000, and we report the fixed and marginal costs that produce it.

Hardware named once. All measurements come from two named cluster nodes with their CPUs and GPUs given, and cross-simulator arms run on a single node because the two nodes carry different CPUs.

Unfavourable results kept. At N=500 the accelerator barely pays for itself, and both cards are slower than the CPU below a few hundred neurons. Those points are in the paper and in the figures.

Maturity: an unsupported topology, a GPU-driver hang beyond a certain workload on one platform, and the absence of head-to-head and realistic-model evaluation.

The head-to-head and realistic-model evaluations are addressed above.

The unsupported topology — a single-compartment branch off a branch point — is detected at initialisation, and that `hsolve` is computed on the CPU, so it costs speed rather than accuracy. This is stated in the manuscript.

The driver hang was investigated and is platform-specific: it occurs on an integrated GPU that also drives the display, and the identical configuration, and far larger ones, run correctly on the A40. The manuscript now says this and names the environment variable that caps model size on such devices. We had removed the discussion in revision; the reviewer was right that it needs an answer rather than silence, and it now has one.

Verification. A reproduction pack is included. `sh reproduce/run_all.sh` rebuilds the software, re-measures the claims on the reader’s own hardware, regenerates the figures from those measurements, and prints each published value beside theirs with a verdict. It refuses to time a binary built for a different GPU than the one in the machine — a failure that produced a 60-fold wrong result for us once and would otherwise cost a reviewer a day.